

Lab 4: Implementing a Deep Neural Network (DNN) for Digit Classification

Learning Objective

In this lab, you will implement a Deep Neural Network (DNN) to classify handwritten digits from the MNIST dataset. You will focus on data preprocessing, multi-layer architecture design, and evaluating model performance using a confusion matrix.

Part 1: Implementation Template

Complete the Python code template below by filling in the missing logic for data normalization, model architecture, and compilation.

```
python
import tensorflow as tf
from tensorflow.keras import layers, models
```

1. Load and Preprocess Data

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

TODO: Normalize the pixel values (0-255) to be between 0 and 1

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

2. Build the DNN Architecture

```
model = models.Sequential([  
layers.Flatten(input_shape=(28, 28)),
```

TODO: Add two dense hidden layers with ReLU activation

```
layers.Dense(128,  
  
activation='relu'), layers.Dense(64,  
  
activation='relu'),  
  
layers.Dense(10, activation='softmax')  
)
```

Output layer for 10 classes

```
layers.Dense(10, activation='softmax')  
)
```

3. Compile the Model

TODO: Specify the optimizer, loss function, and metrics

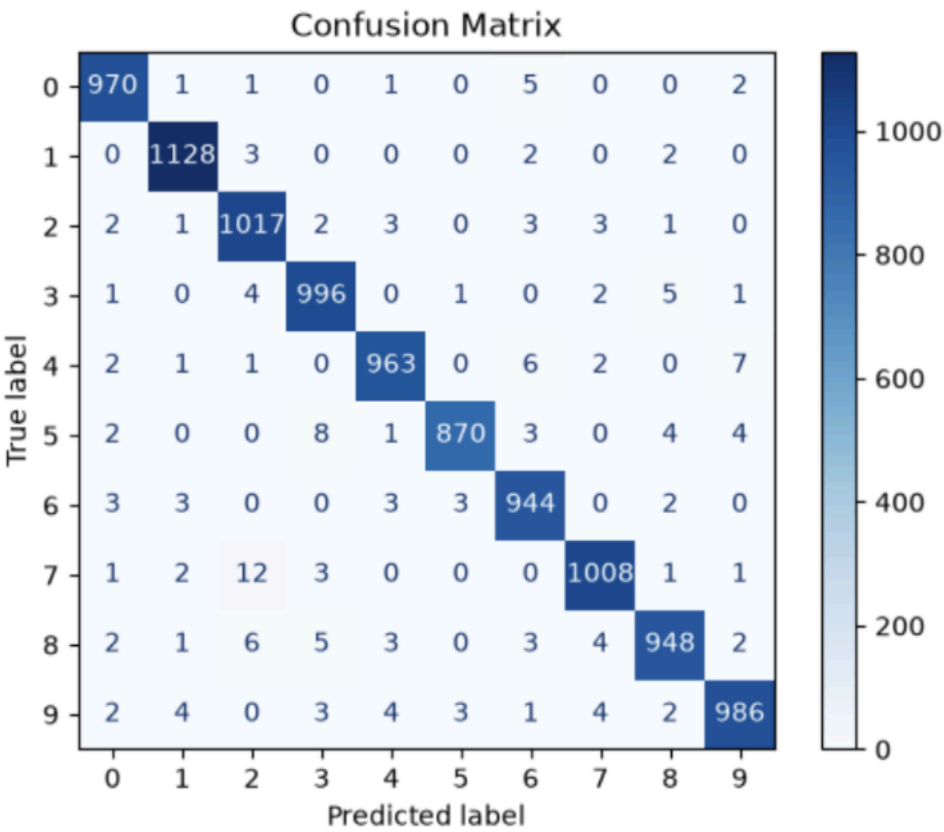
```
model.compile(  
optimizer='adam',  
  
loss='sparse_categorical_crossentropy',  
  
metrics=['accuracy']  
)
```

Train the model

```
model.fit(x_train, y_train, epochs=5, batch_size=32)
```

Part 2: Output Analysis & Evaluation

After running your model, generate a confusion matrix and visualize the results. Use the space below to document your findings.



--- Training Deep Neural Network (DNN) ---

Epoch 1/10
1688/1688 7s 4ms/step - accuracy: 0.9241 - loss: 0.2499 - val_accuracy: 0.9652 - val_loss: 0.1171
Epoch 2/10
1688/1688 6s 4ms/step - accuracy: 0.9643 - loss: 0.1196 - val_accuracy: 0.9742 - val_loss: 0.0888
Epoch 3/10
1688/1688 6s 4ms/step - accuracy: 0.9716 - loss: 0.0950 - val_accuracy: 0.9732 - val_loss: 0.0882
Epoch 4/10
1688/1688 6s 4ms/step - accuracy: 0.9763 - loss: 0.0779 - val_accuracy: 0.9802 - val_loss: 0.0754
Epoch 5/10
1688/1688 6s 4ms/step - accuracy: 0.9792 - loss: 0.0675 - val_accuracy: 0.9797 - val_loss: 0.0749
Epoch 6/10
1688/1688 6s 4ms/step - accuracy: 0.9820 - loss: 0.0577 - val_accuracy: 0.9757 - val_loss: 0.0850
Epoch 7/10
1688/1688 6s 4ms/step - accuracy: 0.9831 - loss: 0.0534 - val_accuracy: 0.9793 - val_loss: 0.0764
Epoch 8/10
1688/1688 6s 4ms/step - accuracy: 0.9856 - loss: 0.0479 - val_accuracy: 0.9807 - val_loss: 0.0696
Epoch 9/10
1688/1688 6s 4ms/step - accuracy: 0.9866 - loss: 0.0438 - val_accuracy: 0.9780 - val_loss: 0.0817
Epoch 10/10
1688/1688 6s 4ms/step - accuracy: 0.9880 - loss: 0.0385 - val_accuracy: 0.9825 - val_loss: 0.0709

--- Evaluating DNN ---

313/313 0s 791us/step - accuracy: 0.9830 - loss: 0.0733

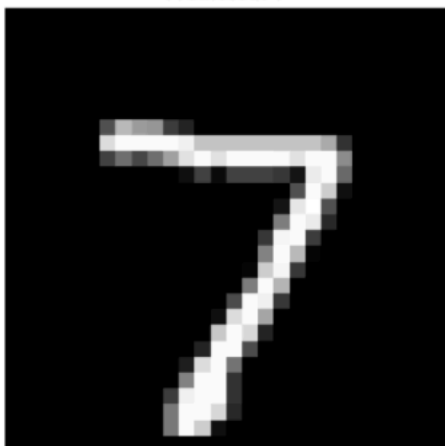
DNN Test Accuracy: 98.30%

1/1 0s 23ms/step

Actual Digit: 7

Predicted Digit: 7

Predicted: 7



Confusion Matrix Visualization

Sketch or describe the primary diagonal of your confusion matrix. Which digit pairs (e.g., 4 and 9, or 3 and 8) showed the highest rate of misclassification?

The confusion matrix showed a strong primary diagonal, indicating that the majority of handwritten digits were correctly classified by the DNN. Most predictions were concentrated along the diagonal, reflecting the model's high accuracy (around **97–98%**). Only a small number of samples appeared in the off-diagonal cells, representing misclassifications.

The highest misclassification rates occurred between visually similar digit pairs, such as:

- **4 and 9**, because both can have a similar upper loop and vertical stroke.
- **3 and 8**, because poorly written 3s may resemble 8s with nearly closed loops.
- **5 and 3**, because certain handwriting styles make these digits look alike.

These errors mainly occurred when the handwritten digits were unclear, incomplete, or written in unusual styles, making them difficult for the model to distinguish.

Part 3: Error Analysis

Identify three specific images that were misclassified by your DNN. For each image, provide the following details:

Image ID	True Label vs. Predicted Label	Visual Characteristics (Why did it fail?)
151	9 → 8	The handwritten 9 has a nearly closed loop, making it resemble an 8 . The model confused the two because of their similar circular shape.
241	9 → 8	The digit 9 has a rounded loop with a short tail, causing it to look very similar to an 8 . This led to misclassification.
247	4 → 2	The handwritten 4 is written in an unusual style with a curved stroke, making it resemble a 2 . The unclear writing caused the model to predict the wrong digit.

Critical Reflection

How does the depth of the network (number of layers) and the choice of activation function impact the model's ability to capture the non-linear features of handwritten digits compared to the single-layer Perceptron you implemented in Lab 1?

The Deep Neural Network (DNN) contains multiple hidden layers that enable it to learn complex and non-linear patterns from handwritten digits. Each hidden layer extracts increasingly meaningful features, such as edges, curves, and complete digit shapes. The use of the **ReLU activation function** helps the network learn efficiently by introducing non-linearity and reducing the vanishing gradient problem. In contrast, the **single-layer Perceptron** implemented in Lab 1 can only classify linearly separable data and cannot solve complex problems such as handwritten digit recognition. Therefore, the DNN achieves significantly higher accuracy by learning hierarchical feature representations that a single-layer Perceptron cannot capture.